

field_results

LOGGER module-attribute

LOGGER = getLogger(__name__)

DatetimeFieldResults

DatetimeFieldResults([field_name](#), [field_results](#), exploration_results)

Bases: [FieldResults](#)

Class containing the results of the exploration compute_stats for a time field.

Parameters:

Name	Type	Description	Default
field_name		field name.	<i>required</i>
field_results		dictionary-like results of the given field obtained from exploration.stats.compute_stats().	<i>required</i>

constant_value instance-attribute

constant_value = _constant_value()

count instance-attribute

count = _stats['count']

field_type instance-attribute

field_type = [field_results](#)[[FIELD_TYPE_KEY](#)]

histogram instance-attribute

histogram = _histogram_df()

histogram_computed instance-attribute

histogram_computed = [HISTOGRAM_KEY](#) in [field_results](#)

histogram_reduced instance-attribute

histogram_reduced = [reduce_histogram](#)(histogram, [result_type](#)=field_type) if histogram_computed else None

is_constant instance-attribute

is_constant = _is_constant()

max instance-attribute [Alt](#)

```
max = _stats['max']
```

min instance-attribute [Alt](#)

```
min = _stats['min']
```

n_missing instance-attribute [Alt](#)

```
n_missing = _stats['missing']
```

n_unhealthy instance-attribute [Alt](#)

```
n_unhealthy = _n_unhealthy()
```

name instance-attribute [Alt](#)

```
name = field_name
```

semantic_tag instance-attribute [Alt](#)

```
semantic_tag = get(SEMANTIC_TAG_KEY)
```

stats instance-attribute [Alt](#)

```
stats = Series(_stats)
```

validations instance-attribute [Alt](#)

```
validations = from_dict(_validations, orient='index', columns=['value', 'alert']) if VALID_KEY in field_results else None
```

plot [Alt](#)

```
plot(time_unit='yearmonthdate', time_type='T', breakdown_by=None, logy=False, interactive=None, mini=False)
```

Altair chart of the field.

Parameters:

Name	Type	Description	Default
time_unit Alt		a time unit supported by Vega-Lite. Recommended: 'yearmonthdate' or 'yearmonthdatehours'. For more details: https://vega.github.io/vega-lite/docs/timeunit.html	'yearmonthdate'
time_type Alt		Vega Data Type for time variable (default: 'T'). Recommended 'T' (= 'temporal') for high granularity time resolution, and 'O' (= 'ordinal') for low cardinality time_unit like 'month'. For more details: https://vega.github.io/vega-lite/docs/type.html	'T'
breakdown_by Alt		str specifying field to breakdown histograms by. Default is None: no breakdown.	None
logy Alt		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
			None

Name	Type	Description	Default
<code>interactive</code> Alt		bool (or None) indicating whether to call the <code>interactive()</code> function on the Altair chart (default: None = False for ordinal <code>time_type</code> otherwise True).	
<code>mini</code> Alt		bool indicating if this is a mini-histogram.	False

Returns:

Type	Description
An Altair chart.	

`plot_calendar` [Alt](#)

```
plot_calendar(xaxis='date', yaxis='yearmonth', color='fr', size='freq', color_scheme=None)
```

Altair chart with calendar-like plot of the field.

Parameters:

Name	Type	Description	Default
<code>xaxis</code> Alt		time unit of the xaxis. Default: 'hours'. Recommended combinations of units are (xaxis='date', yaxis='yearmonth') and (xaxis='hours', yaxis='yearmonthdate')	'date'
<code>yaxis</code> Alt		time unit of the yaxis. Default: 'yearmonthdate'.	'yearmonth'
<code>color</code> Alt		color encoding. It can be either 'freq' (for frequency counts) or 'fr' (for fraud rate). If set to 'fr' then parameters (target_field, target_value) must be given.	'fr'
<code>size</code> Alt		size encoding. It can be either 'freq' (for frequency counts) or 'fr' (for fraud rate). If set to 'fr' then parameters (target_field, target_value) must be given.	'freq'
<code>color_scheme</code> Alt		Vega color scheme to be used. Available schemes can be found here: https://vega.github.io/vega/docs/schemes/ . Default is None, meaning 'blues' for 'freq' and diverging 'blueorange' centered at the overall fraud rate for 'fr'.	None

Returns:

Type	Description
unknown	an Altair chart.

`plot_fraud_rate` [Alt](#)

```
plot_fraud_rate(target=(None, None), time_unit='yearmonthdate', time_type='T', logy=False, interactive=None)
```

Altair fraud rate chart of the field.

Parameters:

Name	Type	Description	Default
<code>target</code> Altair		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. The target_value will be used for the numerator of the fraud rate. Default: (None, None), meaning use the exploration_results.target_field/value if existing.	(None, None)
<code>time_unit</code> Altair		a time unit supported by Vega-Lite. Recommended: 'yearmonthdate' or 'yearmonthdatehours'. For more details: https://vega.github.io/vega-lite/docs/timeunit.html	'yearmonthdate'
<code>time_type</code> Altair		Vega Data Type for time variable (default: 'T'). Recommended 'T' (= 'temporal') for high granularity time resolution, and 'O' (= 'ordinal') for low cardinality time_unit like 'month'. For more details: https://vega.github.io/vega-lite/docs/type.html	'T'
<code>logy</code> Altair		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> Altair		bool (or None) indicating whether to call the interactive() function on the Altair chart (default: None = False for ordinal time_type otherwise True).	None

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal [Altair](#)

```
plot_temporal(chart_type, time_field=None, **kwargs)
```

Altair chart of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>chart_type</code> Altair		type of chart. Options are: '', 'boxplot', 'scatter' and 'missing_values'. can be any of constants.SUPPORTED_AGGREGATIONS, or 'all' meaning that aggregations 'min', 'max' and 'avg' will be overlaid.	<i>required</i>
<code>time_field</code> Altair		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None
<code>**kwargs</code> Altair		plotting options. Look at dedicated plotting functions for reference.	{}

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_aggregation [↗](#)

```
plot_temporal_aggregation(aggregate, time\_field=None, time\_resolution='1D', show\_points=False, interactive=True, zero\_scale=False, logy=False)
```

Altair chart of the field aggregated with respect to time.

Parameters:

Name	Type	Description	Default
time_field ↗		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None
aggregate ↗		aggregation function to be used. Can be any of constants.SUPPORTED_AGGREGATIONS.	<i>required</i>
time_resolution ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here ↗ . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
show_points ↗		bool indicating whether to display the data points (default: False).	False
interactive ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
zero_scale ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: False).	False
logy ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_boxplot [↗](#)

```
plot_temporal_boxplot(time\_field=None, time\_resolution=None, bar\_size=10, interactive=True, zero\_scale=False)
```

Altair boxplot of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: None, meaning that it will be dynamically chosen to divide the time range in 15 periods.	None
<code>bar_size</code> ↗		numeric value used to specify the width of the bars (default: 10).	10
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: False).	False

Returns:

Type	Description
unknown	an Altair chart.

plot_temporal_missing_values [↗](#)

```
plot_temporal_missing_values(time_field=None, time_resolution='1D', show_points=False, interactive=True, zero_scale=True)
```

Altair chart of the missing values of the field of interest over time.

Missing values are entries with value None or that are not of type datetime.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>show_points</code> ↗		bool indicating whether to display the data points (default: False).	False

Name	Type	Description	Default
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
unknown	an Altair chart.

`plot_temporal_scatter` [↗](#)

```
plot_temporal_scatter(time_field=None, breakdown_field=None, interactive=True)
```

Altair scatter plot of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>breakdown_field</code> ↗		name of the field used as a breakdown field. The points will be color encoded by this field.	None
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True

Returns:

Type	Description
unknown	an Altair chart.

`plot_with_fraud_rate` [↗](#)

```
plot_with_fraud_rate(target=(None, None), **kwargs)
```

Altair chart of the field with the fraud rate overlaid.

Parameters:

Name	Type	Description	Default
<code>target</code> ↗		tuple containing (<code>target_field</code> , <code>target_value</code>). Element <code>target_field</code> is the name of the target field. Element <code>target_value</code> indicate the value to be considered as 'positives'. The <code>target_value</code> will be used for the numerator of the fraud rate. Default: (None, None), meaning use the <code>exploration_results.target_field/value</code> if existing.	(None, None)
			{}

Name	Type	Description	Default
**kwargs ¶		plotting options to pass to plot() method.	

Returns:

Type	Description
unknown	an Altair chart.

to_dict [¶](#)

```
to_dict()
```

FieldResults [¶](#)

```
FieldResults(field_name, field_results, exploration_results)
```

Bases: `ABC`

Generic class containing the results of the exploration compute_stats for a particular field.

Parameters:

Name	Type	Description	Default
field_name ¶		field name.	<i>required</i>
field_results ¶		dictionary-like results of the given field obtained from ds_api.exploration.stats.compute_stats .	<i>required</i>

constant_value instance-attribute [¶](#)

```
constant_value = _constant_value()
```

count instance-attribute [¶](#)

```
count = _stats['count']
```

field_type instance-attribute [¶](#)

```
field_type = field_results[FIELD_TYPE_KEY]
```

histogram instance-attribute [¶](#)

```
histogram = _histogram_df()
```

histogram_computed instance-attribute [¶](#)

```
histogram_computed = HISTOGRAM_KEY in field_results
```

histogram_reduced instance-attribute [¶](#)

```
histogram_reduced = reduce_histogram(histogram, result_type=field_type) if histogram_computed else None
```


is_constant instance-attribute [¶](#)

```
is_constant = _is_constant()
```

n_missing instance-attribute [¶](#)

```
n_missing = _stats['missing']
```

n_unhealthy instance-attribute [¶](#)

```
n_unhealthy = _n_unhealthy()
```

name instance-attribute [¶](#)

```
name = field\_name
```

semantic_tag instance-attribute [¶](#)

```
semantic_tag = get(SEMANTIC\_TAG\_KEY)
```

stats instance-attribute [¶](#)

```
stats = Series(_stats)
```

validations instance-attribute [¶](#)

```
validations = from_dict(_validations, orient='index', columns=['value', 'alert']) if VALIDDS\_KEY i  
n field\_results else None
```

plot abstractmethod [¶](#)

```
plot()
```

To be overridden. Altair chart of the field.

plot_fraud_rate abstractmethod [¶](#)

```
plot_fraud_rate(target)
```

To be overridden. Altair fraud rate chart of the field.

plot_temporal abstractmethod [¶](#)

```
plot_temporal(chart_type, time_field, **kwargs)
```

To be overridden. Altair chart of the field over time.

plot_temporal_missing_values abstractmethod [¶](#)

```
plot_temporal_missing_values(time_field)
```

To be overridden. Altair chart of the missing values over time.

plot_with_fraud_rate abstractmethod [¶](#)

```
plot_with_fraud_rate(target)
```

To be overridden. Altair chart of the field with the fraud rate overlaid.

to_dict [¶](#)

```
to_dict()
```

NumericalFieldResults [¶](#)

```
NumericalFieldResults(field\_name, field\_results, exploration_results)
```

Bases: [FieldResults](#)

Class containing the results of the exploration compute_stats for a numerical field.

Parameters:

Name	Type	Description	Default
field_name ¶		field name.	<i>required</i>
field_results ¶		dictionary-like results of the given field obtained from exploration.stats.compute_stats().	<i>required</i>

[bin_width](#) instance-attribute [¶](#)

```
bin_width = _bin_width()
```

[constant_value](#) instance-attribute [¶](#)

```
constant_value = _constant_value()
```

[count](#) instance-attribute [¶](#)

```
count = _stats['count']
```

[field_type](#) instance-attribute [¶](#)

```
field_type = field\_results[FIELD\_TYPE\_KEY]
```

[histogram](#) instance-attribute [¶](#)

```
histogram = _histogram_df()
```

[histogram_computed](#) instance-attribute [¶](#)

```
histogram_computed = HISTOGRAM\_KEY in field\_results
```

[histogram_reduced](#) instance-attribute [¶](#)

```
histogram_reduced = reduce\_histogram(histogram, result\_type=field_type) if histogram_computed else None
```

[is_constant](#) instance-attribute [¶](#)

```
is_constant = _is_constant()
```

[max](#) instance-attribute [¶](#)

```
max = _stats['max']
```

[mean](#) instance-attribute [¶](#)

```
mean = _stats['mean']
```

min instance-attribute [↗](#)

```
min = _stats['min']
```

n_infinities instance-attribute [↗](#)

```
n_infinities = _stats['infinities']
```

n_missing instance-attribute [↗](#)

```
n_missing = _stats['missing']
```

n_nan instance-attribute [↗](#)

```
n_nan = _stats['nan']
```

n_unhealthy instance-attribute [↗](#)

```
n_unhealthy = _n_unhealthy()
```

n_zeros instance-attribute [↗](#)

```
n_zeros = _stats['zeros']
```

name instance-attribute [↗](#)

```
name = field\_name
```

semantic_tag instance-attribute [↗](#)

```
semantic_tag = get(SEMANTIC\_TAG\_KEY)
```

stats instance-attribute [↗](#)

```
stats = Series(_stats)
```

std instance-attribute [↗](#)

```
std = _stats['stddev']
```

sum instance-attribute [↗](#)

```
sum = _stats['sum']
```

validations instance-attribute [↗](#)

```
validations = from_dict(_validations, orient='index', columns=['value', 'alert']) if VALIDDS\_KEY i  
n field\_results else None
```

plot [↗](#)

```
plot(nbins=50, fix\_step=False, normalize=None, breakdown\_by=None, stack=False, logy=False, inter  
active=True, mini=False)
```

Altair chart of the field.

Parameters:

Name	Type	Description	Default
nbins Altair		approximate number of bins for the desired histogram visualization. Default: 50	50
fix_step Altair		if True, constrain the bin width to be a multiple of the original bin width. This ensures that every visual bucket, except for the first and last ones, contain the same amount of original buckets. Otherwise, the binning is automatically decided by Vega (i.e., the binning is optimized to be more pleasing to the human eye). Fixing the step leads to having more control over the desired number of bins, while simultaneously removing undesired bin migration effects. However, it may end up with unpleasant visual bin ranges. If enough resolution was used to calculate the histograms, the mistakes resulting from not fixing the step are likely to be small enough to be safely ignored. Therefore, the option False is preferred. Default is False.	False
normalize Altair		bool (or None) indicating if the histogram is to be normalized. If set as True, the histogram is normalized to unity, otherwise the y-axis represents the actual counts. If set as True and breakdown_by is not None, the histogram of every breakdown value will be normalized to unity. Default is None: this means that the histogram will be normalized if breakdown_by is not None, otherwise it will not be normalized.	None
breakdown_by Altair		string specifying a field to breakdown histograms by. Default is None: no breakdown.	None
stack Altair		whether to stack histograms or not. Default: None. Only applies to the case where breakdown_by is not None.	False
logy Altair		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
interactive Altair		bool indicating whether to call the function interactive() on the Altair chart (default: True).	True
mini Altair		bool indicating if this is a mini-histogram.	False

Returns:

Type	Description
An Altair chart.	

plot_fraud_rate [Altair](#)

```
plot_fraud_rate(target=(None, None), nbins=50, fix_step=False, bar=True, logy=False, interactive=True)
```

Altair fraud rate chart of the field.

Parameters:

Name	Type	Description	Default
<code>target</code> Altair		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. The target_value will be used for the numerator of the fraud rate. Default: (None, None), meaning use the exploration_results.target_field/ value if existing.	(None, None)
<code>bins</code> Altair		approximate number of bins for the desired histogram visualization. Default: 50.	50
<code>fix_step</code> Altair		if True, constrain the bin width to be a multiple of the original bin width. This ensures that every visual bucket, except for the first and last ones, contain the same amount of original buckets. Otherwise, the binning is automatically decided by Vega (i.e., the binning is optimized to be more pleasing to the human eye). Fixing the step leads to having more control over the desired number of bins, while simultaneously removing undesired bin migration effects. However, it may end up with unpleasant visual bin ranges. If enough resolution was used to calculate the histograms, the mistakes resulting from not fixing the step are likely to be small enough to be safely ignored. Therefore, the option False is preferred. Default is False.	False
<code>bar</code> Altair		bool indicating whether to return a bar chart (True) or a line chart (False). Default: True.	True
<code>logy</code> Altair		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> Altair		bool indicating whether to call the function interactive() on the Altair chart (default: True).	True

Returns:

Type	Description
An Altair chart.	

plot_temporal [Altair](#)

```
plot_temporal(chart_type, time_field=None, **kwargs)
```

Altair chart of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>chart_type</code> Altair		type of chart. Options are: "", 'boxplot', 'scatter' and 'missing_values'. can be any of constants.SUPPORTED_AGGREGATIONS, or 'all' meaning that aggregations 'min', 'max' and 'avg' will be overlaid.	<i>required</i>
<code>time_field</code> Altair		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None
		plotting options. Look at dedicated plotting functions for reference.	{}

Name	Type	Description	Default
<code>**kwargs</code> ↗			

Returns:

Type	Description
An Altair chart.	

plot_temporal_aggregation [↗](#)

```
plot_temporal_aggregation(aggregate, time_field=None, time_resolution='1D', show_points=False, interactive=True, zero_scale=False, logy=False)
```

Altair chart of the field aggregated with respect to time.

Parameters:

Name	Type	Description	Default
<code>aggregate</code> ↗		aggregation function to be used. Can be any of constants.SUPPORTED_AGGREGATIONS.	<i>required</i>
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here ↗ . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>show_points</code> ↗		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: False).	False
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False

Returns:

Type	Description

Type	Description
An Altair chart.	

plot_temporal_boxplot [¶](#)

```
plot_temporal_boxplot(time_field=None, time_resolution=None, bar_size=10, interactive=True, zero_scale=False)
```

Altair boxplot of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ¶		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None
<code>time_resolution</code> ¶		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: None, meaning that it will be dynamically chosen to divide the time range in 15 periods.	None
<code>bar_size</code> ¶		numeric value used to specify the width of the bars (default: 10).	10
<code>interactive</code> ¶		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ¶		If true, ensures that a zero baseline value is included in the scale domain. (default: False).	False

Returns:

Type	Description
An Altair chart.	

plot_temporal_missing_values [¶](#)

```
plot_temporal_missing_values(time_field=None, time_resolution='1D', show_points=False, interactive=True, zero_scale=True)
```

Altair chart of the missing values of the field of interest over time.

Missing values are entries with value None or nan.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ¶		name of the field associated with the temporal dimension. If None, the special timestamp_field will be used, if available. Default: None.	None

Name	Type	Description	Default
<code>time_resolution</code> ¶		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>show_points</code> ¶		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> ¶		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ¶		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
An Altair chart.	

`plot_temporal_scatter` [¶](#)

```
plot_temporal_scatter(time_field=None, breakdown_field=None, interactive=True)
```

Altair scatter plot of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ¶		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>breakdown_field</code> ¶		name of the field used as a breakdown field. The points will be color encoded by this field.	None
<code>interactive</code> ¶		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True

Returns:

Type	Description
An Altair chart.	

`plot_with_fraud_rate` [¶](#)

```
plot_with_fraud_rate(target=(None, None), **kwargs)
```

Altair chart of the field with the fraud rate overlaid.

Parameters:

Name	Type	Description	Default
<code>target</code> ¶		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. The target_value will be used for the numerator of the fraud rate. Default: (None, None), meaning use the exploration_results.target_field/value if existing.	(None, None)
<code>**kwargs</code> ¶		plotting options to pass to plot() method.	{}

Returns:

Type	Description
An Altair chart.	

quantile [¶](#)

```
quantile(q=(0.0, 0.25, 0.5, 0.75, 1.0))
```

Return approximate value at the given quantile(s). Values are computed from the histograms, therefore the associated uncertainty is half the bin width of the histogram.

Parameters:

Name	Type	Description	Default
<code>q</code> ¶		float or list/tuple of floats with quantiles to compute. Values must be between 0 and 1. Default is (0.0, 0.25, 0.5, 0.75, 1.0).	(0.0, 0.25, 0.5, 0.75, 1.0)

Returns:

Type	Description
Pandas Series with the quantiles.	

to_dict [¶](#)

```
to_dict()
```

StringFieldResults [¶](#)

```
StringFieldResults(field_name, field_results, exploration_results)
```

Bases: [FieldResults](#)

Class containing the results of the exploration compute_stats for a categorical field.

Parameters:

Name	Type	Description	Default
<code>field_name</code> ¶		field name.	required
<code>field_results</code> ¶		dictionary-like results of the given field obtained from exploration.stats.compute_stats().	required

cardinality instance-attribute [¶](#)

```
cardinality = _cardinality()
```

cardinality_error instance-attribute [¶](#)

```
cardinality_error = _stats['hll_error'] if is_approximate else 0
```

constant_value instance-attribute [¶](#)

```
constant_value = _constant_value()
```

count instance-attribute [¶](#)

```
count = _stats['count']
```

field_type instance-attribute [¶](#)

```
field_type = field_results[FIELD_TYPE_KEY]
```

histogram instance-attribute [¶](#)

```
histogram = _histogram_df()
```

histogram_computed instance-attribute [¶](#)

```
histogram_computed = HISTOGRAM_KEY in field_results
```

histogram_reduced instance-attribute [¶](#)

```
histogram_reduced = reduce_histogram(histogram, result_type=field_type) if histogram_computed else None
```

is_approximate instance-attribute [¶](#)

```
is_approximate = not histogram_computed or OTHER_VAL in [(k[0]) for k in (_histogram)]
```

is_constant instance-attribute [¶](#)

```
is_constant = _is_constant()
```

n_empty instance-attribute [¶](#)

```
n_empty = _stats['empty']
```

n_missing instance-attribute [¶](#)

```
n_missing = _stats['missing']
```

n_unhealthy instance-attribute [¶](#)

```
n_unhealthy = _n_unhealthy()
```

name instance-attribute [¶](#)

```
name = field_name
```

semantic_tag instance-attribute [¶](#)

```
semantic_tag = get(SEMANTIC_TAG_KEY)
```

stats instance-attribute [Altair](#)

```
stats = Series(_stats)
```

validations instance-attribute [Altair](#)

```
validations = from_dict(_validations, orient='index', columns=['value', 'alert']) if VALIDDS_KEY in field_results else None
```

categories [Altair](#)

```
categories(top_n=None)
```

List of top-n most frequent categories ordered by frequency.

Parameters:

Name	Type	Description	Default
<code>top_n</code> Altair		if given return only the top_n most frequent categories.	<code>None</code>

Returns:

Type	Description
<code>list</code>	list of top-n categories ordered by frequency.

frequencies [Altair](#)

```
frequencies(top_n: Optional[int] = None) -> Optional[Series]
```

Frequency table of top-n categories.

Parameters:

Name	Type	Description	Default
<code>top_n</code> Altair	<code>Optional[int]</code>	if given return only the top_n most frequent categories. The rest of the categories will be merge into the '(other)' category.	<code>None</code>

Returns:

Type	Description
<code>Series</code>	Table containing the frequency table of the top_n categories. Indices are category labels, values are frequencies. At most it will have a length of top_n + 2, including the other and the missing keys. If the field histogram was not computed it returns None.

plot [Altair](#)

```
plot(sort_by_freq=True, breakdown_by=None, cat_threshold=None, vertical=True, logy=False, interactive=True, mini=False)
```

Altair chart of the field.

Parameters:

Name	Type	Description	Default
<code>sort_by_freq</code> ¶		bool indicating whether to sort by frequency, i.e. categories with higher frequency first. Otherwise, sort alphabetically (default: True).	True
<code>breakdown_by</code> ¶		str specifying field to breakdown histograms by. Default is None: no breakdown.	None
<code>cat_threshold</code> ¶		number of categories to keep. In case the underlying cardinality is larger, keep the cat_threshold-1 most frequent categories and create a new category 'other' as the aggregation of the rest of categories. Default is None: keep all categories. Only works for string histograms.	None
<code>vertical</code> ¶		bool indicating whether to produce a vertical or a horizontal bar plot. Vertical means categories in the x-axis, counts on the y-axis. Horizontal only possible with no breakdowns. Default: vertical.	True
<code>logy</code> ¶		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> ¶		bool indicating whether to call the function interactive() on the Altair chart. Default: True.	True
<code>mini</code> ¶		bool indicating if this is a mini-histogram.	False

Returns:

Type	Description
An Altair chart.	

`plot_fraud_rate` [¶](#)

```
plot_fraud_rate(target=(None, None), sort_by='fr', cat_threshold=None, vertical=True, logy=False, interactive=True)
```

Altair fraud rate chart of the field.

Parameters:

Name	Type	Description	Default
<code>target</code> ¶		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. The target_value will be used for the numerator of the fraud rate. Default: (None, None), meaning use the exploration_results.target_field/value if existing.	(None, None)
<code>sort_by</code> ¶		string indicating sorting type. Supported sorting are by frequency ('freq'), by fraud rate ('fr') and alphabetical ('alpha'). Default: 'fr'.	'fr'
<code>cat_threshold</code> ¶		number of categories to keep. In case the underlying cardinality is larger, keep the cat_threshold-1 most frequent categories and create a new category 'other' as the aggregation of the rest of categories. Default is None: keep all categories. Only works for string histograms.	None
			True

Name	Type	Description	Default
<code>vertical</code> ↗		bool indicating whether to produce a vertical or a horizontal bar plot. Vertical means categories in the x-axis, counts on the y-axis. Horizontal only possible with no breakdowns. Default: vertical.	
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart. Default: True.	True

Returns:

Type	Description
An Altair chart.	

`plot_temporal` [↗](#)

```
plot_temporal(chart_type, time_field=None, **kwargs)
```

Altair chart of the field with respect to time.

Parameters:

Name	Type	Description	Default
<code>chart_type</code> ↗		type of chart. Options are: 'categories', 'cardinality' and 'missing_values'.	<i>required</i>
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>**kwargs</code> ↗		other options. Look at dedicated plotting functions for reference.	<code>{}</code>

Returns:

Type	Description
An Altair chart.	

`plot_temporal_cardinality` [↗](#)

```
plot_temporal_cardinality(time_field=None, time_resolution='1D', show_points=False, interactive=True, zero_scale=True)
```

Altair chart of the field cardinality with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None

Name	Type	Description	Default
<code>time_resolution</code> Altair		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>show_points</code> Altair		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> Altair		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> Altair		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
An Altair chart.	

`plot_temporal_categories` [Altair](#)

```
plot_temporal_categories(time_field=None, time_resolution='1D', top_n=5, show_points=False, interactive=True, zero_scale=True, logy=False)
```

Altair chart of the categories of a string field with respect to time.

Parameters:

Name	Type	Description	Default
<code>time_field</code> Altair		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>time_resolution</code> Altair		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>top_n</code> Altair		the number of categorical values, with highest count, that should be displayed.	5
<code>show_points</code> Altair		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> Altair		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True

Name	Type	Description	Default
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True
<code>logy</code> ↗		bool indicating whether to use a log10 scale for the y-axis (default: False).	False

Returns:

Type	Description
An Altair chart.	

`plot_temporal_missing_values` [↗](#)

```
plot_temporal_missing_values(time_field=None, time_resolution='1D', show_points=False, interactive=True, zero_scale=True)
```

Altair chart of the missing values of the field of interest over time.

Missing values are entries with value None.

Parameters:

Name	Type	Description	Default
<code>time_field</code> ↗		name of the field associated with the temporal dimension. If None, the special <code>timestamp_field</code> will be used, if available. Default: None.	None
<code>time_resolution</code> ↗		resolution of the time dimension. Available resolutions have to be compliant with the <code>Pandas.GroupBy.freq</code> parameter, more details here <code><https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#offset-aliases></code> . Note that 1) this resolution can not be lower than the intrinsic time resolution of the passed dataframe and 2) for optimal results this resolution should be a multiple of the intrinsic time resolution. The intrinsic resolution is typically specified during the call to <code>compute_stats</code> . Default: '1D'.	'1D'
<code>show_points</code> ↗		bool indicating whether to display the data points (default: False).	False
<code>interactive</code> ↗		bool indicating whether to call the function <code>interactive()</code> on the Altair chart (default: True).	True
<code>zero_scale</code> ↗		If true, ensures that a zero baseline value is included in the scale domain. (default: True).	True

Returns:

Type	Description
An Altair chart.	

`plot_with_fraud_rate` [↗](#)

```
plot_with_fraud_rate(target=(None, None), **kwargs)
```

Altair chart of the field with the fraud rate overlaid.

Parameters:

Name	Type	Description	Default
<code>target</code> ¶		tuple containing (target_field, target_value). Element target_field is the name of the target field. Element target_value indicate the value to be considered as 'positives'. The target_value will be used for the numerator of the fraud rate. Default: (None, None), meaning use the exploration_results.target_field/value if existing.	<code>(None, None)</code>
<code>**kwargs</code> ¶		plotting options to pass to plot() method.	<code>{}</code>

Returns:

Type	Description
<code>unknown</code>	an Altair chart.

`to_dict` [¶](#)

```
to_dict()
```